DOCUMENTO DE ARQUITECTURA & ESPECIFICACIÓN TÉCNICA

REFERENCIA INTEGRAL DE SISTEMAS & OPERACIONES

MANUAL TÉCNICO DE SISTEMAS

Arquitectura Multicapa, Diccionario de Datos Relacional, Máquinas de Estados Transaccionales, Mecanismos Criptográficos de Seguridad, Optimización de Alto Rendimiento e Infraestructura en Producción.

VERSIÓN DE INFRAESTRUCTURA
FusaShop Core v2.4-Production

ESTÁNDAR DE SEGURIDAD
AES-256-CBC • Bcrypt • OWASP Top 10

AUDITORÍA & TRAZABILIDAD
AdminAuditLogs • IntegrationLogs HMAC

GARANTÍA DE RENDIMIENTO
WebP Pipeline • Gzip • Zero N+1 Queries

© 2026 FusaShop — Alcaldía de Fusagasugá & Secretaría de Desarrollo Económico

Documento Técnico Oficial • Nivel de Sensibilidad: Confidencial

Contenido del Manual Técnico

Este documento es la referencia técnica definitiva para Administradores de Sistemas, Oficiales de Seguridad de la Información (CISO), Ingenieros DevOps y Auditores de TI encargados de la supervisión, mantenimiento y escalabilidad de la infraestructura de **FusaShop**.

1. Arquitectura de Software y Topología Multicapa — Capítulo 1
2. Diccionario de Datos y Modelo Entidad-Relación (ERD) — Capítulo 2
3. Máquinas de Estados y Reglas de Transición Transaccional — Capítulo 3
4. Seguridad de la Información, Criptografía y Hardening — Capítulo 4
5. Rendimiento, Optimización de Recursos y Resiliencia — Capítulo 5
6. Despliegue en Producción, Monitoreo y Mantenimiento — Capítulo 6

🔒 **Cumplimiento Normativo y Protección de Datos:** "La plataforma FusaShop cumple con los lineamientos de la Ley Estatutaria 1581 de 2012 de Protección de Datos Personales de Colombia, los estándares de seguridad de pagos PCI-DSS para el aislamiento de credenciales Wompi, y la Resolución DIAN 000042 para facturación electrónica mediante el protocolo Factus."

1. Arquitectura de Software y Modelo Multicapa

FusaShop implementa una arquitectura modular desacoplada en seis capas de abstracción bien definidas, lo que garantiza alta cohesión, bajo acoplamiento y capacidad de escalar horizontalmente:

CAPA 1: EDGE & PRESENTACIÓN (CLIENT LAYER)

`Blade Templates (SSR)` `Tailwind CSS v4 (Compilación JIT Standalone)` `Alpine.js v3 (Micro-reactividad)` `WebP Content Negotiation` `Native Lazy Loading`

CAPA 2: CONTROL, ENRUTAMIENTO & MIDDLEWARES

`SecurityHeaders (CSP / HSTS / X-Frame)` `RoleMiddleware (RBAC)` `EnsureKycApproved` `CompressResponse (Gzip)` `Controllers Segregados: Consumer / Merchant / Analyst`

CAPA 3: SERVICIOS DE DOMINIO & INTELIGENCIA DE NEGOCIO

`CheckoutService` `OrderPaymentService` `StockService (Pessimistic Locking)` `ApiHealthService` `DashboardMetricsService` `TerritoryHeatmapService`

CAPA 4: COLA DE TRABAJOS ASÍNCRONOS (QUEUE / WORKER LAYER)

`ProcessWompiEvent` `GenerateEnviaShipment` `IssueFactusInvoice` `SendLowStockAlert` `SendOrderApprovedEmail`

CAPA 5: PERSISTENCIA, CIFRADO & ALMACENAMIENTO

`Eloquent ORM` `Casts Criptográficos (AES-256)` `MySQL 8.0+ / PostgreSQL 15+` `Redis Cache & Locks` `Storage Disk (WebP Assets)`

CAPA 6: INTEGRACIONES EXTERNAS & API GATEWAY

`Wompi Web Checkout & Webhooks` `Envía Colvanes Logistics REST API` `Factus DIAN Electronic Invoicing` `MCP JSON-RPC 2.0 Server`

1.1 Características Clave de la Arquitectura

- **Desacoplamiento Transaccional:** Las tareas lentas (generación de PDFs con guías de envío, llamadas a la DIAN, envío de correos) nunca ocurren durante el ciclo síncrono HTTP del comprador. Se despachan de inmediato como `Jobs` a la cola gestionada por el worker.

- **Idempotencia en Webhooks:** Las notificaciones asíncronas de Wompi o Envía se verifican contra firmas HMAC SHA-256 y se registran en `integration_logs` antes de ejecutar cualquier mutación de saldo o estado.

2. Diccionario de Datos y Modelo Relacional (ERD)

La base de datos comprende 37 migraciones versionadas. A continuación se detallan las especificaciones técnicas de las tablas y campos más representativos del esquema:

2.1 Tabla `users` (Usuarios y Autenticación)

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN TÉCNICA
`id`	BIGINT UNSIGNED	PK, Auto Increment	Identificador único del usuario en el sistema.
`email`	VARCHAR(255)	Unique, Index	Correo electrónico principal utilizado para login y notificaciones.
`password`	VARCHAR(255)	Not Null	Hash criptográfico de contraseña generado con algoritmo Bcrypt (cost 12).
`role`	ENUM	Default 'consumer'	Roles del sistema: `consumer`, `merchant`, `analyst`.
`loyalty_points`	INTEGER UNSIGNED	Default 0	Saldo activo de FusaCoins acumulados por compras y fidelización.
`verification_code`	VARCHAR(6)	Nullable	Código OTP de 6 dígitos enviado por correo para validación inicial.
`is_blocked`	BOOLEAN	Default false	Flag de suspensión preventiva por auditoría administrativa.

2.2 Tabla `company_profiles` (Comercios & Parámetros Fiscales)

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN TÉCNICA
`id`	BIGINT UNSIGNED	PK, Auto Increment	Identificador del perfil comercial.
`merchant_id`	BIGINT UNSIGNED	FK -> users.id	Relación uno a uno con el usuario propietario de la tienda.
`store_name`	VARCHAR(255)	Not Null	Razón social o nombre público comercial del establecimiento.
`kyc_status`	ENUM	Default 'pending'	Estado de validación legal: `pending`, `approved`, `rejected`.

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN TÉCNICA
`wompi_private_key`	TEXT	Nullable, Cifrado	Llave secreta de Wompi cifrada en base de datos con AES-256-CBC.
`wompi_integrity_secret`	TEXT	Nullable, Cifrado	Secreto de integridad SHA256 para validación de montos.
`latitude / longitude`	DECIMAL(10,8)	Nullable	Coordenadas GPS georreferenciadas para el mapa territorial.

2.3 Tabla `orders` (Órdenes y Transacciones)

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN TÉCNICA
`id`	BIGINT UNSIGNED	PK, Auto Increment	Identificador de la orden.
`reference`	VARCHAR(50)	Unique, Index	Referencia alfanumérica única en formato `FS-YYYYMMDD-XXXX`.
`status`	ENUM	Default 'pending'	Estados: `pending`, `approved`, `processing`, `shipped`, `delivered`, `cancelled`.
`payment_status`	ENUM	Default 'pending'	Estado de pasarela: `pending`, `paid`, `declined`, `voided`, `refunded`.
`total`	DECIMAL(12,2)	Not Null	Monto total a cobrar en pesos colombianos (COP).
`points_discount`	DECIMAL(10,2)	Default 0.00	Descuento económico aplicado por redención de FusaCoins.
`shipping_tracking_number`	VARCHAR(100)	Nullable	Número de guía expedido por la API de Envía Colvanes.
`invoice_number`	VARCHAR(100)	Nullable	Código Único de Factura Electrónica (CUFE) expedido por Factus / DIAN.

2.4 Tabla `admin_audit_logs` (Auditoría Forense)

Proporciona trazabilidad completa e inmutable sobre cualquier mutación crítica realizada desde el portal administrativo:

CAMPO	TIPO DE DATO	DESCRIPCIÓN TÉCNICA
`id`	BIGINT UNSIGNED	Identificador del registro de auditoría.
`admin_id`	BIGINT UNSIGNED	FK -> users.id (identificador del funcionario ejecutor).
`action`	VARCHAR(100)	Acción efectuada (ej: `merchant.kyc.approved`, `merchant.blocked`).
`entity_type`	VARCHAR(100)	Clase Eloquent afectada (ej: `AppModelsCompanyProfile`).
`old_values` / `new_values`	JSON	Instantánea comparativa de los valores modificados para trazabilidad legal.

CAMPO	TIPO DE DATO	DESCRIPCIÓN TÉCNICA
`ip_address`	VARCHAR(45)	Dirección IP IPv4/IPv6 de la estación donde se originó el comando.

3. Máquinas de Estados y Reglas de Transición

FusaShop implementa modelos formales de estado finito para gobernar las órdenes y los trámites de verificación de tiendas, evitando inconsistencias operacionales o estados huérfanos.

3.1 Ciclo de Vida de una Orden de Compra

> **Flujo Secuencial Estándar:**
> `[PENDING]` ➔ Webhook Wompi OK ➔ `[APPROVED]` ➔ Comercio Prepara Paquete ➔ `[PROCESSING]` ➔ Despacho Envía ➔ `[SHIPPED]` ➔ Entrega Confirmada ➔ `[DELIVERED]`

ESTADO ACTUAL	EVENTO DISPARADOR	NUEVO ESTADO	EFECTOS SECUNDARIOS EJECUTADOS
`pending`	Webhook Wompi `APPROVED`	`approved`	Bloqueo de stock confirmado, acreditación de FusaCoins al comprador, encolado de `IssueFactusInvoice` y `GenerateEnviaShipment`.
`pending`	Webhook Wompi `DECLINED`	`cancelled`	Liberación automática del inventario reservado en `StockService` y notificación por correo al comprador.
`approved`	Comercio inicia embalaje	`processing`	Notificación por correo al comprador con detalles del tiempo de alistamiento.
`processing`	Comercio entrega paquete a Envía	`shipped`	Asignación de número de guía y envío de enlace de rastreo en vivo.
`shipped`	Webhook Envía entrega efectiva	`delivered`	Habilitación del formulario de calificación (Review) y liberación de fondos.

3.2 Control de Concurrencia e Idempotencia

Para evitar que un cliente sea cobrado dos veces si Wompi reintenta una llamada de webhook debido a latencia de red:

1. El endpoint `/api/webhooks/wompi` consulta primero la tabla `integration_logs` buscando si el `event_id` ya fue procesado.
2. Se ejecuta una cláusula `Order::where('id', $orderId)->lockForUpdate()` dentro de una transacción de base de datos.
3. Si el estado de la orden ya es `paid` o `approved`, el controlador retorna inmediatamente un código HTTP `200 OK` sin repetir la emisión de factura ni de puntos.

4. Seguridad, Criptografía y Hardening

4.1 Arquitectura Criptográfica

🔐 Cifrado AES-256-CBC en Reposo

Las credenciales privadas de Wompi de los comercios (`wompi_private_key` y `wompi_integrity_secret`) se almacenan cifradas en disco con llave maestra de 256 bits (`APP_KEY`). Nunca se guardan en texto claro.

🔑 Hashing Seguro Bcrypt

Todas las contraseñas de usuarios y analistas se procesan con la función hash adaptativa Bcrypt con un costo de trabajo (work factor) de 12 rondas, mitigando ataques de fuerza bruta.

4.2 Hardening de Cabeceras HTTP (`SecurityHeaders.php`)

El middleware `AppHttpMiddlewareSecurityHeaders` inyecta en cada respuesta HTTP directivas de protección avanzada contra vectores de ataque comunes (XSS, Clickjacking, MIME Sniffing):

```
CABECERAS HTTP INYECTADAS EN FUSASHOP
// Prevención de Clickjacking
X-Frame-Options: SAMEORIGIN

// Prevención de confusión de tipos MIME
X-Content-Type-Options: nosniff

// Control de fuga de información de navegación
Referrer-Policy: strict-origin-when-cross-origin

// Aislamiento de APIs del navegador no autorizadas
Permissions-Policy: camera=(), microphone=(), geolocation=(self)

// Política de Seguridad de Contenido (CSP)
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net; style-sr
```

4.3 Mitigación de Vulnerabilidades OWASP Top 10

- **Inyección SQL:** Mitigada al 100% mediante el uso exclusivo de consultas parametrizadas a través de PDO en Eloquent ORM. No se concatenan variables en consultas crudas.
- **Cross-Site Scripting (XSS):** El motor de plantillas Blade escapa automáticamente todas las variables renderizadas mediante la sintaxis `{{ $variable }}` usando `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`.
- **Cross-Site Request Forgery (CSRF):** Todos los formularios POST, PUT y DELETE requieren la presencia y validación del token `_token` de sesión. Los webhooks externos utilizan firmas HMAC.

5. Rendimiento, Optimización y Resiliencia

5.1 Negociación de Contenido WebP & Compresión Gráfica

Las imágenes representan más del 70% del ancho de banda en un catálogo de comercio electrónico. FusaShop implementa un pipeline automatizado:

🖼️ Conversión Batch a WebP

El script `optimize-images.cjs` comprime sistemáticamente todos los recursos de `public/` y `storage/` logrando reducciones de peso promedio del **74.2%** sin pérdida perceptible de calidad.

📡 Negociación HTTP Transparente

Las rutas de medios inspeccionan la cabecera `Accept: image/webp`. Si el cliente la soporta, el servidor entrega la variante WebP optimizada; en navegadores legados, entrega el PNG original de respaldo.

5.2 Eliminación Radical de Consultas N+1

En pantallas complejas como el panel de reportes analíticos y la auditoría de ventas, se reemplazaron bucles de consultas Eloquent repetitivas por consultas agrupadas SQL con `SUM()` y `GROUP BY`:

```
OPTIMIZACIÓN EN ANALYSTCONTROLLER.PHP — REDUCCIÓN DE 240 A 2 CONSULTAS
// Agrupación en una única consulta indexada
$paymentTotals = DB::table('orders')
    ->whereIn('id', $orderIds)
    ->select('merchant_id', DB::raw('SUM(total) as gross_sales'), DB::raw('COUNT(*) as total_orders'))
    ->groupBy('merchant_id')
    ->pluck('gross_sales', 'merchant_id');
```

5.3 Compresión de Transporte en Servidor (Gzip / Deflate)

Se activaron reglas de compresión en `public/.htaccess` para servidores Apache (`mod_deflate`) y un middleware fallback en PHP (`CompressResponse`):

- Payloads HTML reducidos de ~85 KB a ~16 KB (-81%).
- Hojas de estilo CSS minificadas reducidas de ~42 KB a ~8.5 KB (-79%).
- Respuestas JSON de APIs y FusaBot reducidas en más del 85%.

6. Despliegue en Producción, Monitoreo y Mantenimiento

6.1 Dimensionamiento Mínimo de Servidor

RECURSO	REQUISITO MÍNIMO	RECOMENDADO PARA PRODUCCIÓN
Sistema Operativo	Ubuntu Server 22.04 LTS / 24.04 LTS	Ubuntu Server 24.04 LTS con Kernel optimizado
CPU	2 vCPU (2.4 GHz+)	4 vCPU dedicadas
Memoria RAM	2 GB RAM + 2 GB Swap	8 GB RAM (con Redis in-memory)
Almacenamiento	25 GB SSD	80 GB NVMe con backups automáticos

6.2 Configuración de Workers con Supervisor

Para asegurar que las colas de correos, facturación Factus y guías de Envía se procesen continuamente sin interrupción, se configura el demonio Supervisor en `/etc/supervisor/conf.d/fusashop-worker.conf`:

CONFIGURACIÓN DE SUPERVISOR EN LINUX

```
[program:fusashop-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/fusashop/artisan queue:work --sleep=3 --tries=3 --max-time=3600 --timeout=90
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/fusashop/storage/logs/worker.log
stopwaitsecs=3600
```

6.3 Cron Job para Laravel Scheduler

En el archivo crontab del usuario del servidor web (`crontab -e -u www-data`), registrar una única entrada para gestionar tareas recurrentes (limpieza de carritos abandonados, expiración de banners, sondeo de salud de APIs):

LLAMADA MINUTAL AL SCHEDULER DE LARAVEL

```
* * * * * cd /var/www/fusashop && php artisan schedule:run >> /dev/null 2>&1
```

6.4 Check-List de Pase a Producción (Go-Live)

1. Ejecutar `php artisan config:cache`, `php artisan route:cache` y `php artisan view:cache`.

2. Establecer `APP_ENV=production` y `APP_DEBUG=false` en el archivo `.env`.
3. Configurar certificado SSL/TLS con cifrado fuerte (Let's Encrypt / Certbot con renovación automática).
4. Verificar que las llaves de pasarela de los comercios en `company_profiles` estén cifradas.
5. Ejecutar `php artisan test` para validar que los 76 tests pasan limpiamente antes de abrir tráfico público.